LAPORAN TUGAS
PEMROGRAMAN BERBASIS OBJEK

SISTEM RUMAH SAKIT

Implementasi Inheritance dan Polymorphism dengan C# (.NET)

Nama	:	.....
NIM	:	.....
Kelas	:	A
Link Repository GitHub	:	.....

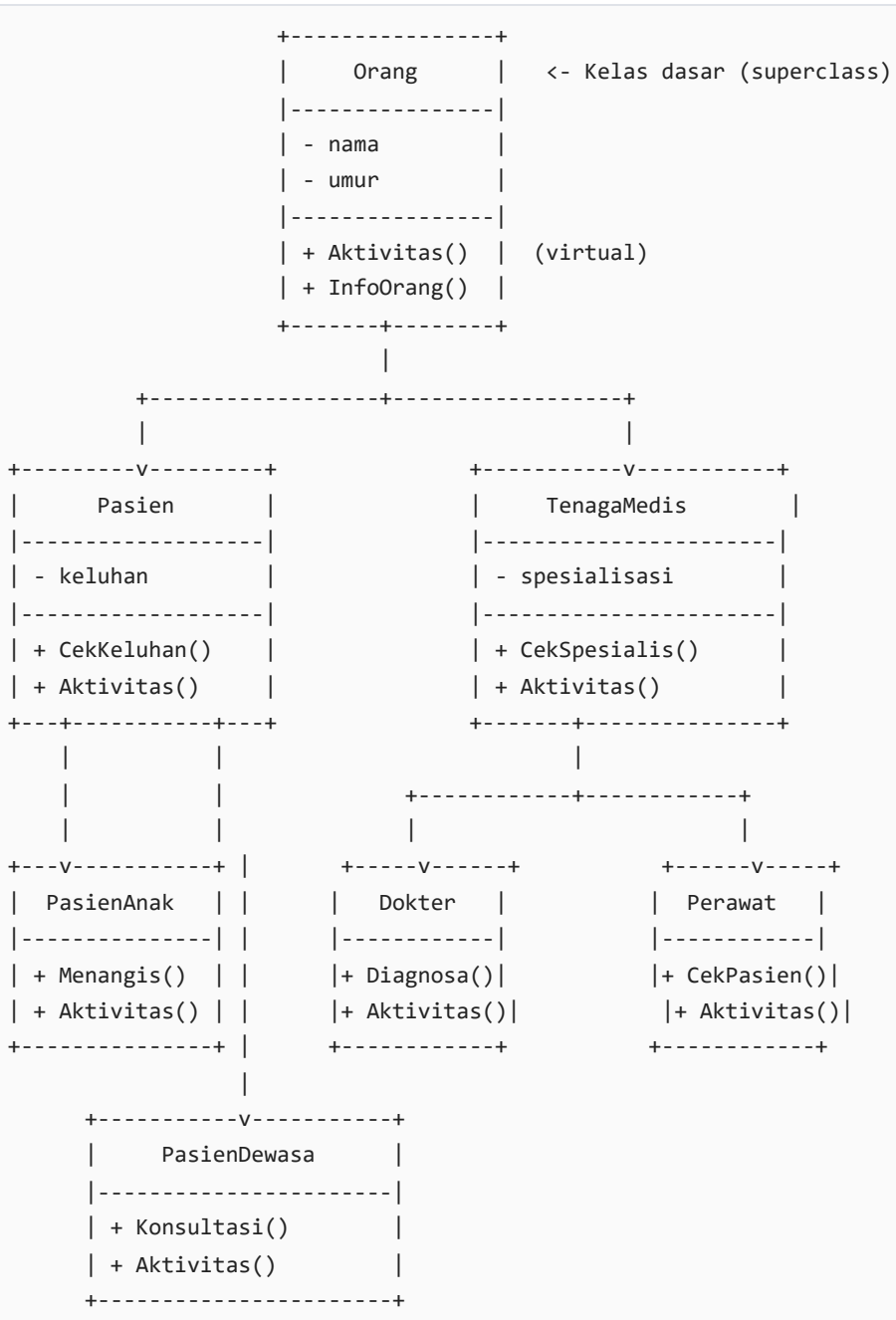
Dibuat dengan Visual Studio / .NET SDK 10 • Bahasa pemrograman C#
Repository: https://github.com/GylaxDev/A_252410103018_TASKPBO (public) • Link Pengumpulan: https://forms.gle/5KVcRH9JYByq7soc8

BAB I PENDAHULUAN

Laporan ini menyajikan implementasi program **Sistem Rumah Sakit** menggunakan bahasa pemrograman **C#** pada platform .NET. Program dibuat untuk memenuhi instruksi tugas Pemrograman Berbasis Objek (PBO), yaitu mengimplementasikan hierarki kelas, pewarisan (*inheritance*), *override* method, *polymorphism*, serta koleksi objek.

Struktur program terdiri dari kelas dasar **Orang**, kelas turunan **Pasien** dan **TenagaMedis**, kelas turunan lanjutan (**Dokter**, **Perawat**, **PasienAnak**, **PasienDewasa**), serta kelas **RumahSakit** yang menyimpan koleksi Orang. Method **Main()** mendemonstrasikan pembuatan objek, penambahan objek ke rumah sakit, penampilan data, polymorphism, dan pemanggilan method khusus setiap kelas.

A. Hierarki Kelas (Class Hierarchy)



Keterangan:
- Semua subclass mewarisi properti Nama dan Umur dari kelas Orang.
- Method Aktivitas() di-override di setiap subclass -> polymorphism.
- Kelas RumahSakit menyimpan koleksi bertipe Orang sehingga dapat menampung objek Dokter, Perawat, PasienAnak, dan PasienDewasa secara bersama-sama.

B. Daftar Kelas, Properti, dan Method

Kelas	Induk	Properti Tambahan	Method Khusus (selain warisan)
`Orang`	`-`	`Nama, Umur`	`Aktivitas() (virtual), InfoOrang()`
`Pasien`	`Orang`	`Keluhan`	`CekKeluhan(), Aktivitas() override`
`TenagaMedis`	`Orang`	`Spesialisasi`	`CekSpesialis(), Aktivitas() override`
`Dokter`	`TenagaMedis`	`-`	`Diagnosa(string), Aktivitas() override`
`Perawat`	`TenagaMedis`	`-`	`CekPasien(string), Aktivitas() override`
`PasienAnak`	`Pasien`	`-`	`Menangis(), Aktivitas() override`
`PasienDewasa`	`Pasien`	`-`	`Konsultasi(string), Aktivitas() override`
`RumahSakit`	`-`	`List<Orang>`	`TambahOrang(Orang), DaftarOrang()`

Catatan: Semua subclass mewarisi properti **Nama** dan **Umur** dari kelas **Orang**. Method **Aktivitas()** di-override di setiap subclass sehingga menghasilkan output berbeda sesuai objek sebenarnya (*polymorphism*). Kelas **RumahSakit** menyimpan koleksi `List<Orang>` sehingga dapat menampung objek Dokter, Perawat, PasienAnak, dan PasienDewasa secara bersama-sama.

BAB II KODE PROGRAM

1. Kelas Orang (Induk / Superclass)

```
namespace SistemRumahSakit;

/// <summary>
/// Kelas dasar Orang (Induk dari semua kelas dalam sistem rumah sakit).
/// </summary>
public class Orang
{
    public string Nama { get; set; }
    public int Umur { get; set; }

    public Orang(string nama, int umur)
    {
        Nama = nama;
        Umur = umur;
    }

    /// <summary>
    /// Method aktivitas umum. Bersifat virtual agar bisa di-override oleh subclass.
    /// </summary>
    public virtual void Aktivitas()
    {
        Console.WriteLine($"{Nama} ({Umur} tahun) sedang beraktivitas umum.");
    }

    /// <summary>
    /// Menampilkan informasi dasar orang.
    /// </summary>
    public void InfoOrang()
    {
        Console.WriteLine($"Nama: {Nama} | Umur: {Umur}");
    }
}
```

2. Kelas Pasien (Mewarisi Orang)

```
namespace SistemRumahSakit;

/// <summary>
/// Kelas Pasien mewarisi kelas Orang.
/// </summary>
public class Pasien : Orang
{
    public string Keluhan { get; set; }

    public Pasien(string nama, int umur, string keluhan)
        : base(nama, umur)
    {
        Keluhan = keluhan;
    }
```

```
15
16      /// <summary>
17      /// Method khusus kelas Pasien untuk menampilkan keluhan.
18      /// </summary>
19      public void CekKeluhan()
20      {
21          Console.WriteLine($"{Nama} memiliki keluhan: {Keluhan}");
22      }
23
24      public override void Aktivitas()
25      {
26          Console.WriteLine($"{Nama} (pasien) sedang beristirahat di kamar rumah sakit.");
27      }
28  }
```

3. Kelas TenagaMedis (Mewarisi Orang)

```
1   namespace SistemRumahSakit;
2
3   /// <summary>
4   /// Kelas TenagaMedis mewarisi kelas Orang.
5   /// </summary>
6   public class TenagaMedis : Orang
7   {
8       public string Spesialisasi { get; set; }
9
10      public TenagaMedis(string nama, int umur, string spesialisasi)
11          : base(nama, umur)
12      {
13          Spesialisasi = spesialisasi;
14      }
15
16      /// <summary>
17      /// Method khusus kelas TenagaMedis untuk menampilkan spesialisasi.
18      /// </summary>
19      public void CekSpesialis()
20      {
21          Console.WriteLine($"{Nama} memiliki spesialisasi: {Spesialisasi}");
22      }
23
24      public override void Aktivitas()
25      {
26          Console.WriteLine($"{Nama} (tenaga medis) sedang bertugas di rumah sakit.");
27      }
28  }
```

4. Kelas Dokter (Mewarisi TenagaMedis)

```
1   namespace SistemRumahSakit;
2
3   /// <summary>
4   /// Kelas Dokter mewarisi kelas TenagaMedis.
5   /// </summary>
6   public class Dokter : TenagaMedis
```

```
{
    public Dokter(string nama, int umur, string spesialisasi)
        : base(nama, umur, spesialisasi)
    {
    }

    /// <summary>
    /// Method khusus kelas Dokter untuk mendiagnosa pasien.
    /// </summary>
    public void Diagnosa(string namaPasien)
    {
        Console.WriteLine($"Dokter {Nama} ({Spesialisasi}) sedang mendiagnosa pasien: {namaPasien}")
        ;
    }

    public override void Aktivitas()
    {
        Console.WriteLine($"Dokter {Nama} sedang memeriksa pasien di ruang praktik.");
    }
}
```

5. Kelas Perawat (Mewarisi TenagaMedis)

```
namespace SistemRumahSakit;

/// <summary>
/// Kelas Perawat mewarisi kelas TenagaMedis.
/// </summary>
public class Perawat : TenagaMedis
{
    public Perawat(string nama, int umur, string spesialisasi)
        : base(nama, umur, spesialisasi)
    {
    }

    /// <summary>
    /// Method khusus kelas Perawat untuk memeriksa keadaan pasien.
    /// </summary>
    public void CekPasien(string namaPasien)
    {
        Console.WriteLine($"Perawat {Nama} ({Spesialisasi}) sedang memeriksa keadaan pasien: {namaPa
        sien}");
    }

    public override void Aktivitas()
    {
        Console.WriteLine($"Perawat {Nama} sedang merawat dan memantau pasien.");
    }
}
```

6. Kelas PasienAnak (Mewarisi Pasien)

```
namespace SistemRumahSakit;

```

```
/// <summary>
/// Kelas PasienAnak mewarisi kelas Pasien.
/// </summary>
public class PasienAnak : Pasien
{
    public PasienAnak(string nama, int umur, string keluhan)
        : base(nama, umur, keluhan)
    {
    }

    /// <summary>
    /// Method khusus kelas PasienAnak.
    /// </summary>
    public void Menangis()
    {
        Console.WriteLine($"{Nama} (pasien anak) sedang menangis: \"Waaa... waaa...\"");
    }

    public override void Aktivitas()
    {
        Console.WriteLine($"{Nama} (pasien anak) sedang bermain di ruang bermain rumah sakit.");
    }
}
```

7. Kelas PasienDewasa (Mewarisi Pasien)

```
namespace SistemRumahSakit;

/// <summary>
/// Kelas PasienDewasa mewarisi kelas Pasien.
/// </summary>
public class PasienDewasa : Pasien
{
    public PasienDewasa(string nama, int umur, string keluhan)
        : base(nama, umur, keluhan)
    {
    }

    /// <summary>
    /// Method khusus kelas PasienDewasa.
    /// </summary>
    public void Konsultasi(string namaDokter)
    {
        Console.WriteLine($"{Nama} (pasien dewasa) sedang berkonsultasi dengan {namaDokter}.");
    }

    public override void Aktivitas()
    {
        Console.WriteLine($"{Nama} (pasien dewasa) sedang menunggu di ruang tunggu rumah sakit.");
    }
}
```

8. Kelas RumahSakit (Koleksi Orang)

```
namespace SistemRumahSakit;

/// <summary>
/// Kelas RumahSakit yang menyimpan koleksi (list) objek Orang.
/// </summary>
public class RumahSakit
{
    private readonly List<Orang> daftarOrang = new();

    /// <summary>
    /// Menambahkan sebuah objek Orang ke dalam rumah sakit.
    /// </summary>
    public void TambahOrang(Orang orang)
    {
        daftarOrang.Add(orang);
        Console.WriteLine($"{orang.Nama} telah ditambahkan ke data rumah sakit.");
    }

    /// <summary>
    /// Menampilkan seluruh data orang yang terdaftar di rumah sakit.
    /// </summary>
    public void DaftarOrang()
    {
        Console.WriteLine();
        Console.WriteLine("=== DAFTAR SELURUH ORANG DI RUMAH SAKIT ===");
        foreach (var orang in daftarOrang)
        {
            orang.InfoOrang();
        }
        Console.WriteLine("==========================================");
    }

    /// <summary>
    /// Mengakses koleksi orang (untuk demonstrasi polymorphism).
    /// </summary>
    public List<Orang> GetSemuaOrang() => daftarOrang;
}
```

9. Program.cs — Method Main() dan Demonstrasi Soal

```
using System.Text;
using SistemRumahSakit;

Console.OutputEncoding = Encoding.UTF8;

Console.WriteLine("=========================================================");
Console.WriteLine("        SISTEM RUMAH SAKIT - TUGAS PBO");
Console.WriteLine("=========================================================");
Console.WriteLine();

// ---------------------------------------------------------------
// 6a. Buat objek rumah sakit
```

```
// ------------------------------------------------------------
var rumahSakit = new RumahSakit();

// ------------------------------------------------------------
// 6b. Buat beberapa objek orang
// ------------------------------------------------------------
var dokter1 = new Dokter("Dr. Andi Prasetyo", 45, "Jantung");
var dokter2 = new Dokter("Dr. Siti Rahayu", 38, "Anak");
var perawat1 = new Perawat("Budi Santoso", 30, "Perawatan Umum");
var pasienAnak1 = new PasienAnak("Timmy", 7, "Demam");
var pasienDewasa1 = new PasienDewasa("Rina Wijaya", 40, "Hipertensi");
var pasienUmum1 = new Pasien("Joko Susilo", 55, "Sakit Gigi");

Console.WriteLine(">>> Objek-objek telah dibuat.\n");

// ------------------------------------------------------------
// 6c. Tambahkan ke rumah sakit
// ------------------------------------------------------------
Console.WriteLine(">>> Menambahkan objek ke rumah sakit:");
rumahSakit.TambahOrang(dokter1);
rumahSakit.TambahOrang(dokter2);
rumahSakit.TambahOrang(perawat1);
rumahSakit.TambahOrang(pasienAnak1);
rumahSakit.TambahOrang(pasienDewasa1);
rumahSakit.TambahOrang(pasienUmum1);

// ------------------------------------------------------------
// 6d. Tampilkan semua data
// ------------------------------------------------------------
rumahSakit.DaftarOrang();

// ------------------------------------------------------------
// 6e. Demonstrasikan polymorphism
//     Memanggil Aktivitas() yang sama pada variabel bertipe Orang,
//     namun hasilnya berbeda-beda sesuai tipe objek sebenarnya.
// ------------------------------------------------------------
Console.WriteLine("\n>>> Demonstrasi Polymorphism:");
Console.WriteLine("Memanggil Aktivitas() melalui variabel bertipe Orang:");
foreach (var orang in rumahSakit.GetSemuaOrang())
{
    Console.Write($"[{orang.GetType().Name}] ".PadRight(20));
    orang.Aktivitas();
}

Console.WriteLine("\n>>> Demonstrasi Polymorphism dengan ARRAY bertipe Orang:");
Orang[] kumpulanOrang = { dokter1, perawat1, pasienAnak1, pasienDewasa1 };
foreach (var orang in kumpulanOrang)
{
    orang.Aktivitas();
}

// ------------------------------------------------------------
// 6f. Panggil method khusus
// ------------------------------------------------------------
Console.WriteLine("\n>>> Memanggil method khusus setiap kelas:");
```

```
dokter1.CekSpesialis();
dokter2.CekSpesialis();
perawat1.CekSpesialis();
pasienAnak1.CekKeluhan();
pasienDewasa1.CekKeluhan();
pasienUmum1.CekKeluhan();

Console.WriteLine("\n>>> Method khusus Dokter & Perawat:");
dokter1.Diagnosa(pasienUmum1.Nama);
perawat1.CekPasien(pasienAnak1.Nama);

Console.WriteLine("\n>>> Method khusus PasienAnak & PasienDewasa:");
pasienAnak1.Menangis();
pasienDewasa1.Konsultasi(dokter2.Nama);

// ================================================================
// JAWABAN SOAL-SOAL (dengan demonstrasi pada method Main)
// ================================================================

// ----------------------------------------------------------------
// SOAL 1: Jalankan method Aktivitas() pada objek Dokter dan Pasien.
//          Apa yang terjadi? Mengapa demikian?
// ----------------------------------------------------------------
Console.WriteLine("\n\n================ SOAL 1 ================");
Console.WriteLine("Menjalankan Aktivitas() pada objek Dokter dan Pasien:");
dokter1.Aktivitas();
pasienUmum1.Aktivitas();
Console.WriteLine(">> Yang terjadi: masing-masing objek menjalankan versi Aktivitas()");
Console.WriteLine("   milik kelasnya sendiri (override), bukan versi kelas Orang.");
Console.WriteLine(">> Mengapa: Aktivitas() di kelas Orang bersifat 'virtual' dan");
Console.WriteLine("   di-override di setiap subclass, sehingga C# memilih versi");
Console.WriteLine("   method sesuai tipe objek sebenarnya (runtime polymorphism).");

// ----------------------------------------------------------------
// SOAL 2: Jalankan method Diagnosa() pada objek Dokter.
//          Apa yang terjadi? Mengapa demikian?
// ----------------------------------------------------------------
Console.WriteLine("\n================ SOAL 2 ================");
Console.WriteLine("Menjalankan Diagnosa() pada objek Dokter:");
dokter1.Diagnosa(pasienUmum1.Nama);
Console.WriteLine(">> Yang terjadi: method Diagnosa() berjalan normal dan menampilkan");
Console.WriteLine("   informasi diagnosis.");
Console.WriteLine(">> Mengapa: Diagnosa() adalah method yang didefinisikan di kelas");
Console.WriteLine("   Dokter, dan variabel dokter1 bertipe Dokter, sehingga method");
Console.WriteLine("   tersebut bisa langsung diakses.");

// ----------------------------------------------------------------
// SOAL 3: Dapatkah Anda menampilkan informasi lengkap
//          (nama, umur, spesialisasi) dari objek Dokter?
//          Bagaimana caranya?
// ----------------------------------------------------------------
Console.WriteLine("\n================ SOAL 3 ================");
Console.WriteLine("Ya, bisa. Informasi lengkap objek Dokter dapat ditampilkan dengan");
Console.WriteLine("menggabungkan InfoOrang() (warisan Orang) dan CekSpesialis()");
Console.WriteLine("(warisan TenagaMedis). Contoh:");
```

```
123
124 // Cara 1: memakai method yang sudah ada
125 dokter1.InfoOrang();
126 dokter1.CekSpesialis();
127
128 // Cara 2: memakai variabel bertipe Orang (melalui downcast)
129 Console.WriteLine("\nCara lain dengan variabel bertipe Orang (via downcast):");
130 Orang orangDokter = dokter1;
131 orangDokter.InfoOrang();
132 ((TenagaMedis)orangDokter).CekSpesialis();
133
134 // Cara 3: mengakses properti secara langsung
135 Console.WriteLine("\nCara lain dengan mengakses properti secara langsung:");
136 Console.WriteLine($"Nama: {dokter1.Nama} | Umur: {dokter1.Umur} | Spesialisasi: {dokter1.Spesialisas
137         i}");
138
139 // ------------------------------------------------------------
140 // SOAL 4: Jalankan method Menangis() pada objek PasienAnak.
141 //          Apa yang terjadi? Mengapa demikian?
142 // ------------------------------------------------------------
143 Console.WriteLine("\n================ SOAL 4 ================");
144 Console.WriteLine("Menjalankan Menangis() pada objek PasienAnak:");
145 pasienAnak1.Menangis();
146 Console.WriteLine(">> Yang terjadi: method Menangis() berjalan normal dan menampilkan");
147 Console.WriteLine("   aktivitas menangis dari pasien anak.");
148 Console.WriteLine(">> Mengapa: Menangis() didefinisikan di kelas PasienAnak, dan objek");
149 Console.WriteLine("   pasienAnak1 memang bertipe PasienAnak, sehingga method tersebut");
150 Console.WriteLine("   bisa diakses langsung.");
151
152 // ------------------------------------------------------------
153 // SOAL 5: Buatlah variabel bertipe Orang, lalu isi dengan objek
154 //          Perawat. Jalankan method Aktivitas(). Apa yang terjadi?
155 //          Mengapa demikian?
156 // ------------------------------------------------------------
157 Console.WriteLine("\n================ SOAL 5 ================");
158 Orang orangPerawat = new Perawat("Dewi Lestari", 35, "Kebidanan");
159 Console.WriteLine("Variabel bertipe Orang diisi dengan objek Perawat.");
160 Console.WriteLine("Menjalankan Aktivitas():");
161 orangPerawat.Aktivitas();
162 Console.WriteLine(">> Yang terjadi: yang dijalankan adalah Aktivitas() versi Perawat,");
163 Console.WriteLine("   BUKAN versi kelas Orang.");
164 Console.WriteLine(">> Mengapa: karena Aktivitas() bersifat 'virtual' dan telah");
165 Console.WriteLine("   di-override oleh kelas Perawat, C# melakukan dynamic dispatch");
166 Console.WriteLine("   (pemanggilan method ditentukan oleh tipe objek saat runtime,");
167 Console.WriteLine("   bukan tipe variabel saat compile time).");
168 Console.WriteLine(">> Catatan: method khusus Perawat (misal CekPasien) TIDAK bisa");
169 Console.WriteLine("   dipanggil langsung dari variabel bertipe Orang; perlu downcast:");
170
171 // Menunjukkan bahwa method khusus butuh downcast
172 // orangPerawat.CekPasien("X");  // <- ini akan ERROR compile
173 ((Perawat)orangPerawat).CekPasien("Timmy");
174 Console.WriteLine("   (baris orangPerawat.CekPasien(...) akan menghasilkan error");
175 Console.WriteLine("    compiler karena tipe variabel hanya Orang)");
176
177 // ------------------------------------------------------------
```

```
178 // SOAL 6: Kesimpulan apa yang dapat Anda ambil?
179 // ------------------------------------------------------------
180 Console.WriteLine("\n================ SOAL 6 ================");
181 Console.WriteLine("KESIMPULAN:");
182 Console.WriteLine("1. Inheritance (pewarisan) memungkinkan subclass (Dokter, Perawat,");
183 Console.WriteLine("   Pasien, dll.) menggunakan kembali properti dan method dari");
184 Console.WriteLine("   kelas induk Orang, sehingga lebih sederhana dan terstruktur.");
185 Console.WriteLine("2. Polymorphism (virtual + override) membuat method yang sama,");
186 Console.WriteLine("   misalnya Aktivitas(), dapat berperilaku berbeda sesuai objek");
187 Console.WriteLine("   sebenarnya meskipun dipanggil lewat variabel bertipe induk.");
188 Console.WriteLine("3. Variabel bertipe induk (Orang) bisa menyimpan objek subclass");
189 Console.WriteLine("   apa pun, sehingga memudahkan pengelolaan koleksi orang di");
190 Console.WriteLine("   rumah sakit (misalnya List<Orang>).");
191 Console.WriteLine("4. Method khusus tiap subclass hanya bisa diakses bila tipe variabel");
192 Console.WriteLine("   sesuai dengan kelas tempat method itu didefinisikan, atau");
193 Console.WriteLine("   menggunakan downcast ((Subkelas)variabel).");
194
195 Console.WriteLine("\n================ SELESAI ================");
```

BAB III OUTPUT PROGRAM

Berikut adalah hasil keluaran program ketika dijalankan (`dotnet run`). Output ini memperlihatkan seluruh langkah yang diminta: pembuatan objek rumah sakit, pembuatan objek, penambahan objek, penampilan data, demonstrasi polymorphism, pemanggilan method khusus, serta demonstrasi untuk menjawab Soal 1 sampai Soal 6.

```
=======================================================
        SISTEM RUMAH SAKIT - TUGAS PBO
=======================================================

>>> Objek-objek telah dibuat.

>>> Menambahkan objek ke rumah sakit:
Dr. Andi Prasetyo telah ditambahkan ke data rumah sakit.
Dr. Siti Rahayu telah ditambahkan ke data rumah sakit.
Budi Santoso telah ditambahkan ke data rumah sakit.
Timmy telah ditambahkan ke data rumah sakit.
Rina Wijaya telah ditambahkan ke data rumah sakit.
Joko Susilo telah ditambahkan ke data rumah sakit.

=== DAFTAR SELURUH ORANG DI RUMAH SAKIT ===
Nama: Dr. Andi Prasetyo | Umur: 45
Nama: Dr. Siti Rahayu | Umur: 38
Nama: Budi Santoso | Umur: 30
Nama: Timmy | Umur: 7
Nama: Rina Wijaya | Umur: 40
Nama: Joko Susilo | Umur: 55
=========================================

>>> Demonstrasi Polymorphism:
Memanggil Aktivitas() melalui variabel bertipe Orang:
[Dokter]            Dokter Dr. Andi Prasetyo sedang memeriksa pasien di ruang praktik.
[Dokter]            Dokter Dr. Siti Rahayu sedang memeriksa pasien di ruang praktik.
[Perawat]           Perawat Budi Santoso sedang merawat dan memantau pasien.
[PasienAnak]        Timmy (pasien anak) sedang bermain di ruang bermain rumah sakit.
[PasienDewasa]      Rina Wijaya (pasien dewasa) sedang menunggu di ruang tunggu rumah sakit.
[Pasien]            Joko Susilo (pasien) sedang beristirahat di kamar rumah sakit.

>>> Demonstrasi Polymorphism dengan ARRAY bertipe Orang:
Dokter Dr. Andi Prasetyo sedang memeriksa pasien di ruang praktik.
Perawat Budi Santoso sedang merawat dan memantau pasien.
Timmy (pasien anak) sedang bermain di ruang bermain rumah sakit.
Rina Wijaya (pasien dewasa) sedang menunggu di ruang tunggu rumah sakit.

>>> Memanggil method khusus setiap kelas:
Dr. Andi Prasetyo memiliki spesialisasi: Jantung
Dr. Siti Rahayu memiliki spesialisasi: Anak
Budi Santoso memiliki spesialisasi: Perawatan Umum
Timmy memiliki keluhan: Demam
Rina Wijaya memiliki keluhan: Hipertensi
Joko Susilo memiliki keluhan: Sakit Gigi

```

```
>>> Method khusus Dokter & Perawat:
Dokter Dr. Andi Prasetyo (Jantung) sedang mendiagnosa pasien: Joko Susilo
Perawat Budi Santoso (Perawatan Umum) sedang memeriksa keadaan pasien: Timmy

>>> Method khusus PasienAnak & PasienDewasa:
Timmy (pasien anak) sedang menangis: "Waaa... waaa..."
Rina Wijaya (pasien dewasa) sedang berkonsultasi dengan Dr. Siti Rahayu.


================ SOAL 1 ================
Menjalankan Aktivitas() pada objek Dokter dan Pasien:
Dokter Dr. Andi Prasetyo sedang memeriksa pasien di ruang praktik.
Joko Susilo (pasien) sedang beristirahat di kamar rumah sakit.
>> Yang terjadi: masing-masing objek menjalankan versi Aktivitas()
   milik kelasnya sendiri (override), bukan versi kelas Orang.
>> Mengapa: Aktivitas() di kelas Orang bersifat 'virtual' dan
   di-override di setiap subclass, sehingga C# memilih versi
   method sesuai tipe objek sebenarnya (runtime polymorphism).

================ SOAL 2 ================
Menjalankan Diagnosa() pada objek Dokter:
Dokter Dr. Andi Prasetyo (Jantung) sedang mendiagnosa pasien: Joko Susilo
>> Yang terjadi: method Diagnosa() berjalan normal dan menampilkan
   informasi diagnosis.
>> Mengapa: Diagnosa() adalah method yang didefinisikan di kelas
   Dokter, dan variabel dokter1 bertipe Dokter, sehingga method
   tersebut bisa langsung diakses.

================ SOAL 3 ================
Ya, bisa. Informasi lengkap objek Dokter dapat ditampilkan dengan
menggabungkan InfoOrang() (warisan Orang) dan CekSpesialis()
(warisan TenagaMedis). Contoh:
Nama: Dr. Andi Prasetyo | Umur: 45
Dr. Andi Prasetyo memiliki spesialisasi: Jantung

Cara lain dengan variabel bertipe Orang (via downcast):
Nama: Dr. Andi Prasetyo | Umur: 45
Dr. Andi Prasetyo memiliki spesialisasi: Jantung

Cara lain dengan mengakses properti secara langsung:
Nama: Dr. Andi Prasetyo | Umur: 45 | Spesialisasi: Jantung

================ SOAL 4 ================
Menjalankan Menangis() pada objek PasienAnak:
Timmy (pasien anak) sedang menangis: "Waaa... waaa..."
>> Yang terjadi: method Menangis() berjalan normal dan menampilkan
   aktivitas menangis dari pasien anak.
>> Mengapa: Menangis() didefinisikan di kelas PasienAnak, dan objek
   pasienAnak1 memang bertipe PasienAnak, sehingga method tersebut
   bisa diakses langsung.

================ SOAL 5 ================
Variabel bertipe Orang diisi dengan objek Perawat.
Menjalankan Aktivitas():
Perawat Dewi Lestari sedang merawat dan memantau pasien.
```

```
>> Yang terjadi: yang dijalankan adalah Aktivitas() versi Perawat,
   BUKAN versi kelas Orang.
>> Mengapa: karena Aktivitas() bersifat 'virtual' dan telah
   di-override oleh kelas Perawat, C# melakukan dynamic dispatch
   (pemanggilan method ditentukan oleh tipe objek saat runtime,
   bukan tipe variabel saat compile time).
>> Catatan: method khusus Perawat (misal CekPasien) TIDAK bisa
   dipanggil langsung dari variabel bertipe Orang; perlu downcast:
Perawat Dewi Lestari (Kebidanan) sedang memeriksa keadaan pasien: Timmy
   (baris orangPerawat.CekPasien(...) akan menghasilkan error
    compiler karena tipe variabel hanya Orang)

================ SOAL 6 ================
KESIMPULAN:
1. Inheritance (pewarisan) memungkinkan subclass (Dokter, Perawat,
   Pasien, dll.) menggunakan kembali properti dan method dari
   kelas induk Orang, sehingga lebih sederhana dan terstruktur.
2. Polymorphism (virtual + override) membuat method yang sama,
   misalnya Aktivitas(), dapat berperilaku berbeda sesuai objek
   sebenarnya meskipun dipanggil lewat variabel bertipe induk.
3. Variabel bertipe induk (Orang) bisa menyimpan objek subclass
   apa pun, sehingga memudahkan pengelolaan koleksi orang di
   rumah sakit (misalnya List<Orang>).
4. Method khusus tiap subclass hanya bisa diakses bila tipe variabel
   sesuai dengan kelas tempat method itu didefinisikan, atau
   menggunakan downcast ((Subkelas)variabel).

================ SELESAI ================
```

BAB IV JAWABAN SOAL-SOAL

Pertanyaan 1	
Jalankan method Aktivitas() pada objek Dokter dan Pasien. Apa yang terjadi? Mengapa demikian?	**Yang terjadi:** method Aktivitas() yang dijalankan adalah versi *override* milik kelas **Dokter** (menampilkan "Dokter ... sedang memeriksa pasien di ruang praktik") dan versi *override* milik kelas **Pasien** ("... sedang beristirahat di kamar rumah sakit"), **bukan** versi dari kelas Orang. **Mengapa demikian:** karena method Aktivitas() pada kelas Orang dideklarasikan dengan keyword **virtual**, lalu di-*override* di setiap subclass. Saat program dijalankan, C# melakukan *dynamic dispatch* (late binding): pemilihan method dilakukan berdasarkan **tipe objek sebenarnya saat runtime**, bukan tipe variabelnya. Inilah konsep **polymorphism** (banyak rupa) dalam PBO.

Pertanyaan 2	
Jalankan method Diagnosa() pada objek Dokter. Apa yang terjadi? Mengapa demikian?	**Yang terjadi:** method Diagnosa() berjalan normal dan menampilkan informasi bahwa dokter sedang mendiagnosa pasien. **Mengapa demikian:** karena method Diagnosa() didefinisikan langsung di dalam kelas **Dokter**, dan objek yang digunakan (dokter1) bertipe Dokter. Karena tipe variabel sama dengan kelas tempat method didefinisikan, method tersebut dapat dipanggil langsung. Sebaliknya, jika objek tersebut disimpan dalam variabel bertipe Orang atau TenagaMedis, maka Diagnosa() **tidak bisa** dipanggil tanpa *downcast*, misalnya: `((Dokter)variabel).Diagnosa(...)`.

Pertanyaan 3	
Dapatkah Anda menampilkan informasi lengkap (nama, umur, spesialisasi) dari objek Dokter? Bagaimana caranya?	**Ya, bisa.** Caranya sebagai berikut: 1. Memanggil method `InfoOrang()` (warisan dari kelas Orang) untuk menampilkan nama dan umur, lalu memanggil `CekSpesialis()` (warisan dari kelas TenagaMedis) untuk menampilkan spesialisasi. 2. Mengakses properti secara langsung: `dokter.Nama`, `dokter.Umur`, dan `dokter.Spesialisasi`. 3. Jika objek disimpan dalam variabel bertipe Orang, gunakan *downcast*, misalnya `((TenagaMedis)orang).CekSpesialis()`. 4. Alternatif lain: membuat method baru, misalnya `InfoLengkap()`, di kelas Dokter yang menggabungkan seluruh informasi tersebut dalam satu method.

Pertanyaan 4	
Jalankan method Menangis() pada objek PasienAnak. Apa	**Yang terjadi:** method Menangis() berjalan normal dan menampilkan aktivitas menangis dari pasien anak.

yang terjadi? Mengapa demikian?	**Mengapa demikian:** karena method Menangis() didefinisikan di kelas **PasienAnak**, dan objek yang digunakan bertipe PasienAnak, sehingga objek langsung memiliki akses ke method tersebut. Jika objek disimpan dalam variabel bertipe Pasien atau Orang, method Menangis() tidak bisa dipanggil tanpa *downcast* ke PasienAnak.

Pertanyaan 5	
Buatlah variabel bertipe Orang, lalu isi dengan objek Perawat. Jalankan method Aktivitas(). Apa yang terjadi? Mengapa demikian?	**Yang terjadi:** yang dijalankan adalah Aktivitas() versi **Perawat** ("Perawat ... sedang merawat dan memantau pasien"), **BUKAN** versi kelas Orang. **Mengapa demikian:** variabel bertipe Orang diperbolehkan menyimpan objek subclass apa pun (*upcasting*). Karena Aktivitas() bersifat **virtual** dan telah di-*override* oleh kelas Perawat, C# melakukan *dynamic dispatch*: method yang dipilih ditentukan oleh **tipe objek sebenarnya saat runtime** (Perawat), bukan tipe variabel saat compile time (Orang). **Catatan:** method khusus Perawat seperti `CekPasien()` tidak dapat dipanggil langsung dari variabel bertipe Orang; harus menggunakan *downcast*: `((Perawat)orang).CekPasien(...)`.

Pertanyaan 6	
Kesimpulan apa yang dapat Anda ambil?	1. **Inheritance (pewarisan)** memungkinkan subclass menggunakan kembali properti dan method dari superclass, sehingga program lebih ringkas, terstruktur, dan mudah dikembangkan. 2. **Polymorphism (virtual + override)** membuat method yang sama dapat berperilaku berbeda sesuai objek sebenarnya, meskipun dipanggil melalui variabel bertipe induk. Contohnya `List<Orang>` dapat menampung objek Dokter, Perawat, dan Pasien sekaligus. 3. Method khusus subclass hanya dapat diakses bila tipe variabel sesuai dengan kelas tempat method itu didefinisikan, atau dengan menggunakan *downcast*. 4. Pewarisan bertingkat (multilevel inheritance) memperjelas hierarki sistem: Orang → TenagaMedis → Dokter/Perawat, dan Orang → Pasien → PasienAnak/PasienDewasa.

BAB V PENUTUP

Seluruh instruksi tugas telah diimplementasikan pada program Sistem Rumah Sakit: kelas-kelas beserta properti dan method-nya, *override* method Aktivitas() di setiap subclass, koleksi Orang pada kelas RumahSakit, serta method Main() yang mendemonstrasikan pembuatan objek, penambahan objek, penampilan data, polymorphism, dan pemanggilan method khusus. Demonstrasi untuk menjawab seluruh pertanyaan (Soal 1–6) juga disertakan di dalam program dan ditampilkan pada output BAB III laporan ini.

Project diunggah ke repository GitHub **A_252410103018_TASKPBO** dengan akses **public** sesuai instruksi pengumpulan (format KELAS_NIM_TASKPBO): https://github.com/GylaxDev/A_252410103018_TASKPBO, dan modul disimpan dalam format PDF agar siap dikumpulkan melalui tautan: https://forms.gle/5KVcRH9JYByq7soc8